

Lecture 6: Policy Optimization II: State of the Art

Bolei Zhou

<https://github.com/zhoubolei/introRL>

July 4, 2020

Today's Plan

- ① Variants of policy gradient algorithms
- ② Two lines of works on SOTA policy optimization:
 - ① Policy Gradient→Natural policy gradient/TRPO→ACKTR→PPO
 - ② Q-learning→DDPG→TD3→SAC

Value-based RL versus Policy-based RL

- 1 In value-based RL deterministic policy is generated directly from the value function using greedy $a_t = \arg \max_a Q(a, s_t)$
- 2 In policy optimization, we have stochastic policy output from $\pi_\theta(a|s)$ where θ is the policy parameter to optimize
- 3 The policy objective is

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

- 4 The policy gradient (REINFORCE) is

$$\begin{aligned}\nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta}[G_t \nabla_\theta \log \pi_\theta(s, a)] \\ &\approx \frac{1}{N} \sum_{i=1}^N G_t^i \nabla_\theta \log \pi_\theta(s_i, a_i)\end{aligned}$$

Reducing the Variance of Actor-Critic by a Baseline

- 1 Advantage function: combine Q with baseline V

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

- 2 Then the policy gradient becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(s, a) A^{\pi_\theta}(s, a)]$$

Advantage Actor-Critic

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) A^{\pi_{\theta}}(s, a)] \\ A^{\pi}(s, a) &= Q^{\pi}(s, a) - V^{\pi}(s)\end{aligned}$$

- 1 Two function approximators contain two sets of parameter vectors,

$$\begin{aligned}V_{\mathbf{v}}(s) &\approx V^{\pi}(s) \\ Q_{\mathbf{w}}(s, a) &\approx Q^{\pi}(s, a)\end{aligned}$$

- 2 Updating both parameters by TD learning or MC

Advantage Actor-Critic

- ① For the true value function $V^{\pi_\theta}(s)$, the TD error δ^{π_θ}

$$\delta^{\pi_\theta} = r(s, a) + \gamma V^{\pi_\theta}(s') - V^{\pi_\theta}(s)$$

is an estimate of the advantage function

$$\begin{aligned}\mathbb{E}_{\pi_\theta}[\delta^{\pi_\theta} | s, a] &= \mathbb{E}_{\pi_\theta}[r + \gamma V^{\pi_\theta}(s') | s, a] - V^{\pi_\theta}(s) \\ &= Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s) \\ &= A^{\pi_\theta}(s, a)\end{aligned}$$

- ② So we can use the TD error to compute the policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s, a) \delta^{\pi_\theta}]$$

- ③ It requires only **one set of critic parameter κ** that is estimated by TD

$$\delta_v = r + \gamma V_\kappa(s') - V_\kappa(s)$$

Critic at Different Time-Scales

- 1 Critic can estimate linear value function $V_{\kappa}(s) = \psi(s)^T \kappa$ from many targets as follows
- 2 For MC, the update is

$$\Delta\kappa = \alpha(\textcolor{red}{G}_t - V_{\kappa}(s))\psi(s)$$

- 3 For TD(0), the update is

$$\Delta\kappa = \alpha(\textcolor{red}{r} + \gamma V_{\kappa}(s') - V_{\kappa}(s))\psi(s)$$

- 4 For k-step return, the update is

$$\Delta\kappa = \alpha\left(\sum_{i=0}^{\textcolor{red}{k}} \gamma^i \textcolor{red}{r}_{t+i} + \gamma^{\textcolor{red}{k}} V_{\kappa}(s_{t+k}) - V_{\kappa}(s_t)\right)\psi(s)$$

Actors at Different Time-Scales

- ① The policy gradient can also be estimated at many time-scales

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) A^{\pi_{\theta}}(s, a)]$$

- ② Monte-Carlo Actor-Critic policy gradient uses error from complete return

$$\nabla_{\theta} J(\theta) = \alpha (G_t - V_{\kappa}(s_t)) \nabla_{\theta} \log \pi_{\theta}(s_t, a_t)$$

- ③ TD Difference Actor-Critic policy gradient uses TD error

$$\nabla_{\theta} J(\theta) = \alpha (r + \gamma V_{\kappa}(s_{t+1}) - V_{\kappa}(s_t)) \nabla_{\theta} \log \pi_{\theta}(s_t, a_t)$$

- ④ k-step return Actor-Critic policy gradient uses the k-step return error

$$\nabla_{\theta} J(\theta) = \alpha \left(\sum_{i=0}^k \gamma^i r_{t+i} + \gamma^k V_{\kappa}(s_{t+k}) - V_{\kappa}(s_t) \right) \nabla_{\theta} \log \pi_{\theta}(s_t, a_t)$$

Summary of Policy Gradient Algorithms

- 1 The policy gradient has many forms

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) G_t] \text{ — REINFORCE} \\ &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) Q^w(s, a)] \text{ — Q Actor-Critic} \\ &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) A^w(s, a)] \text{ — Advantage Actor-Critic} \\ &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \delta] \text{ — TD Actor-Critic}\end{aligned}$$

- 2 Critic uses policy evaluation (e.g. MC or TD learning) to estimate $Q^{\pi}(s, a)$, $A^{\pi}(s, a)$, or $V^{\pi}(s)$

The State of the Art RL Methods

Two lines of works on policy optimization:

- ① Policy Gradient → Natural Policy Gradient/TRPO → ACKTR → PPO
- ② Q-learning → DDPG → TD3 → SAC

State of the Art (SOTA) for Policy Optimization

① Policy Gradient→Natural PG/TRPO→ACKTR→PPO

- ① **TRPO**: Trust region policy optimization. Schulman, L., Moritz, Jordan, Abbeel. 2015
- ② **ACKTR**: Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation. Y. Wu, E. Mansimov, S. Liao, R. Grosse, and J. Ba. 2017
- ③ **PPO**: Proximal policy optimization algorithms. Schulman, Wolski, Dhariwal, Radford, Klimov. 2017

② Q-learning→DDPG→TD3→SAC

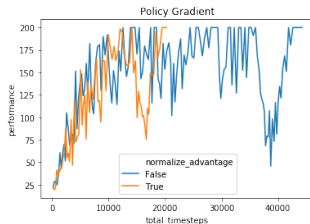
- ① **DDPG**: Deterministic Policy Gradient Algorithms, Silver et al. 2014
- ② **TD3**: Addressing Function Approximation Error in Actor-Critic Methods, Fujimoto et al. 2018
- ③ **SAC**: Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor, Haarnoja et al. 2018

Problems with Policy Gradient

- 1 Poor sample efficiency as PG is on-policy learning,

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) r(s, a)]$$

- 2 Large policy update or improper step size destroy the training
 - 1 This is different from supervised learning where the learning and data are independent
 - 2 In RL, step too far $\rightarrow$ bad policy $\rightarrow$ bad data collection
 - 3 May not be able to recover from a bad policy, which collapses the overall performance



Problems with Policy Gradient

- ① How to make the training more stable?
 - ① Solution: Trust region and natural policy gradient
- ② How to make it as off-policy policy optimization
 - ① Solution: Importance sampling used in TRPO

Natural Policy Gradient

- 1 Policy gradient is the steepest ascend in **parameter space** with Euclidean metric:

$$d^* = \nabla_{\theta} J(\theta) = \lim_{\epsilon \rightarrow 0} \frac{1}{\epsilon} \arg \max J(\theta + d), s.t. \|d\| \leq \epsilon$$

- 1 Drawback: it is sensitive to the parameterization of the policy function
- 2 Steepest ascend in **distribution space (policy output)** with KL-divergence as constraint

$$d^* = \arg \max J(\theta + d), s.t. KL(\pi_{\theta} || \pi_{\theta+d}) = c \quad (1)$$

- 1 Fixing KL-divergence as a constant c makes sure we move along the distribution space with constant speed, regardless the curvature (thus robust to the parametrization of the model as we only care about the distribution induced by the parameter)

KL-Divergence as Metric for Two Distributions

- 1 KL-divergence measure the “closeness” of two distributions.

$$KL(\pi_{\theta}||\pi_{\theta'}) = E_{\pi_{\theta}}[\log \pi_{\theta}] - E_{\pi_{\theta}}[\log \pi_{\theta'}]$$

- 2 Although as KL-divergence is non-symmetric and thus not a true metric, we can use it anyway. This is because as d goes to zero, KL-divergence is asymptotically symmetric. So, within a local neighbourhood, KL-divergence is approximately symmetric
- 3 We can prove that the second-order Taylor expansion of KL-divergence is

$$KL(\pi_{\theta}||\pi_{\theta+d}) \approx \frac{1}{2}d^T F d$$

where F is the **Fisher Information Matrix** as second order derivative of KL divergence $E_{\pi_{\theta}}[\nabla \log \pi_{\theta} \nabla \log \pi_{\theta}^T]$

Natural Policy Gradient

$$d^* = \arg \max J(\theta + d), s.t. KL[\pi_\theta || \pi_{\theta+d}] = c$$

- 1 Write the above as Lagrangian form, with the $J(\theta + d)$ approximated by its first order Taylor expansion and the constraint KL-divergence approximated by its second order Taylor expansion

$$\begin{aligned} d^* &= \arg \max_d J(\theta + d) - \lambda(KL(\pi_\theta || \pi_{\theta+d}) - c) \\ &\approx \arg \max_d J(\theta) + \nabla_\theta J(\theta)^T d - \frac{1}{2} \lambda d^T F d + \lambda c \end{aligned}$$

- 2 To solve this maximization we set its derivative wrt. d to zero, then we have the **natural policy gradient**: $d = \frac{1}{\lambda} F^{-1} \nabla_\theta J(\theta)$

Natural Policy Gradient

- 1 Natural policy gradient is a **second-order** optimization that is more accurate and works regardless of how the model is parameterized (model invariant).

$$\theta_{t+1} = \theta_t + \alpha F^{-1} \nabla_{\theta} J(\theta)$$

- 1 where $F = E_{\pi_{\theta}(s,a)}[\nabla \log \pi_{\theta}(s,a) \nabla \log \pi_{\theta}(s,a)^T]$ is the Fisher information matrix, also as the second-order derivative of the KL-divergence
- 2 F measures the curvature of the policy(distribution) relative to the model parameter θ
- 2 Natural policy gradient produces the same policy change **regardless of the model parameterization**.
- 3 A detailed read on natural gradient: <https://wiseodd.github.io/techblog/2018/03/14/natural-gradient/>

Policy Gradient with Importance Sampling

- 1 We can turn PG into off-policy learning using importance sampling
- 2 **Importance sampling**: IS calculates the expected value of $f(x)$ where x has a data distribution p
 - 1 we can sample data from another distribution q and use the probability ratio between p and q to re-calibrate the result

$$\mathbb{E}_{x \sim p}[f(x)] = \int p(x)f(x)dx = \int q(x)\frac{p(x)}{q(x)}f(x)dx = \mathbb{E}_{x \sim q}\left[\frac{p(x)}{q(x)}f(x)\right]$$

- 2 Code example of IS: [link](#)
- 3 Using important sampling in policy objective

$$J(\theta) = \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}}[r(s, \mathbf{a})] = \mathbb{E}_{\mathbf{a} \sim \hat{\pi}}\left[\frac{\pi_{\theta}(s, \mathbf{a})}{\hat{\pi}(s, \mathbf{a})}r(s, \mathbf{a})\right]$$

Increasing the Robustness with Trust Regions

- 1 The behavior policy could be the old policy directly, thus we can have a surrogate objective function

$$\theta = \arg \max_{\theta} J_{\theta_{old}}(\theta) = \arg \max_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} R_t \right]$$

- 2 The estimate might be excessively large when $\pi_{\theta}/\pi_{\theta_{old}}$ is too large
- 3 Solution: to limit the difference between subsequent policies
- 4 For instance, use the Kullbeck-Leibler (KL) divergence to measure the distance between two policies

$$KL(\pi_{\theta_{old}}||\pi_{\theta}) = - \sum_a \pi_{\theta_{old}}(a|s) \log \frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)}$$

Increasing the Robustness with Trust Regions

- ① Thus our objective with trust region becomes, to maximize

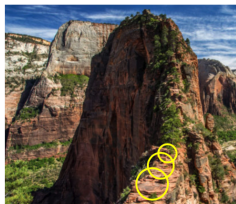
$$J_{\theta_{old}}(\theta) = \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} R_t \right]$$

$$\text{subject to } KL(\pi_{\theta_{old}}(\cdot|s_t) || \pi_{\theta}(\cdot|s_t)) \leq \delta$$

- ② In the trust region, we limit our parameter search within a region controlled by δ . This is the intuition behind algorithms TRPO and PPO



Gradient ascent



Trust region

Trust Region Optimization

- 1 Following Taylor's series expansion on both terms above up to the second-order
- 2 After some derivations we can have

$$J_{\theta_t}(\theta) \approx g^T(\theta - \theta_t)$$
$$KL(\theta_t || \theta) \approx \frac{1}{2}(\theta - \theta_t)^T H(\theta - \theta_t)$$

where $g = \nabla_{\theta} J_{\theta_t}(\theta)$ and $H = \nabla_{\theta}^2 KL(\theta_t || \theta)$ and θ_t is the old policy parameter

- 3 Then the objective turns to:

$$\theta_{t+1} = \arg \max_{\theta} g^T(\theta - \theta_t) \text{ s.t. } \frac{1}{2}(\theta - \theta_t)^T H(\theta - \theta_t) \leq \delta$$

- 4 This is a quadratic equation and can be solved analytically:

$$\theta_{t+1} = \theta_t + \sqrt{\frac{2\delta}{g^T H^{-1} g}} H^{-1} g$$

Natural Policy Gradient

- 1 Natural gradient is the steepest ascent direction with respect to the Fisher information

$$\theta_{t+1} = \theta_t + \sqrt{\frac{2\delta}{g^T H^{-1} g}} H^{-1} g$$

- 2 H is the Fisher Information Matrix (FIM) which can be computed explicitly as

$$H = \nabla_{\theta}^2 KL(\pi_{\theta_t} || \pi_{\theta}) = E_{a,s \sim \pi_{\theta_t}} \left[\nabla_{\theta} \log \pi_{\theta}(a, s) \nabla_{\theta} \log \pi_{\theta}(a, s)^T \right]$$

- 3 Learning rate (δ) can be thought of as choosing a step size that is normalized **with respect to the change in the policy**
- 4 This is beneficial because it means that any parameter update won't significantly change the output of the policy network

Natural Policy Gradient part of TRPO

Algorithm 1 Natural Policy Gradient

Input: initial policy parameters θ_0

for $k = 0, 1, 2, \dots$ **do**

Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Form sample estimates for

- policy gradient $\hat{g}_k$ (using advantage estimates)
- and KL-divergence Hessian / Fisher Information Matrix $\hat{H}_k$

Compute Natural Policy Gradient update:

$$\theta_{k+1} = \theta_k + \sqrt{\frac{2\delta}{\hat{g}_k^T \hat{H}_k^{-1} \hat{g}_k}} \hat{H}_k^{-1} \hat{g}_k$$

end for

① Sham Kakade. “A Natural Policy Gradient.” NIPS 2001

Trust Region Policy Optimization (TRPO)

- 1 FIM and its inverse are very expensive to compute
- 2 TRPO estimates the term $x = H^{-1}g$ by solving the following linear equation $Hx = g$
- 3 Consider the optimization for a quadratic equation

Solving $Ax = b$ is equivalent to

$$x = \arg \max_x f(x) = \frac{1}{2}x^T Ax - b^T x$$

$$\text{since } f'(x) = Ax - b = 0$$

- 4 Thus we can optimize the quadratic equation as

$$\min_x \frac{1}{2}x^T Hx - g^T x$$

- 5 Use conjugate gradient method to solve it. It is very similar to the gradient ascent but can be done in fewer iterations

Trust Region Policy Optimization (TRPO)

- ① Resulting algorithm is a refined version of natural policy gradient

Algorithm 3 Trust Region Policy Optimization

Input: initial policy parameters θ_0

for $k = 0, 1, 2, \dots$ **do**

Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Form sample estimates for

- policy gradient $\hat{g}_k$ (using advantage estimates)
- and KL-divergence Hessian-vector product function $f(v) = \hat{H}_k v$

Use CG with n_{cg} iterations to obtain $x_k \approx \hat{H}_k^{-1} \hat{g}_k$

Estimate proposed step $\Delta_k \approx \sqrt{\frac{2\delta}{x_k^T \hat{H}_k x_k}} x_k$

Perform backtracking line search with exponential decay to obtain final update

$$\theta_{k+1} = \theta_k + \alpha^j \Delta_k$$

end for

Trust Region Policy Optimization (TRPO)

- ① Schulman, et al. ICML 2015: a lot of proofs
- ② The appendix A of the TRPO paper provides a 2-page proof that establishes the guaranteed monotonic improvement that **the policy update in each TRPO iteration creates a better policy**

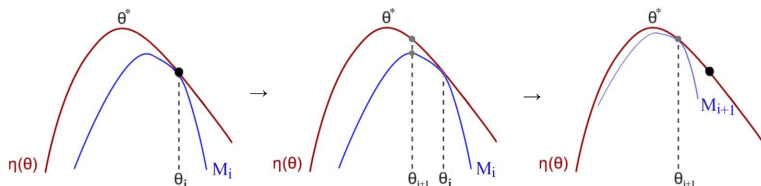
$$J(\pi_{t+1}) - J(\pi_t) \geq M_t(\pi_{t+1}) - M(\pi_t)$$

where $M_t(\pi) = L_{\pi_t}(\pi) - \alpha D_{KL}(\pi_t, \pi)$

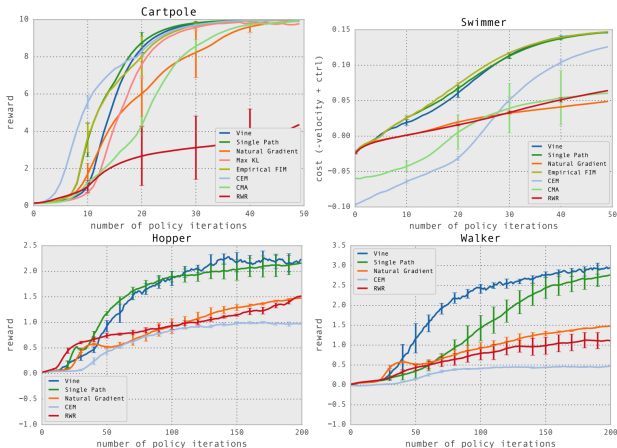
- ③ Thus by maximizing M_t at each iteration, we guarantee that the true objective J is non-decreasing

Trust Region Policy Optimization (TRPO)

- 1 It is a type of Minorize-Maximization (MM) algorithm which is a class of methods that includes expectation maximization
- 2 The MM algorithm achieves this iteratively by maximizing surrogate function (the blue line below) approximating the expected reward locally.



Result and Demo of TRPO



1 Demo video is at
<https://www.youtube.com/watch?v=KJ15iGGJFvQ>

Limitations of TRPO

1 Scalability issue for TRPO

- 1 Computing H every time for the current policy model is expensive
- 2 It requires a large batch of rollouts to approximate H .

$$H = E_{s,a \sim \pi_{\theta_t}} \left[(\nabla_{\theta} \log \pi_{\theta}(s, a))^T (\nabla_{\theta} \log \pi_{\theta}(s, a)) \right]$$

3 Conjugate Gradient(CG) makes implementation more complicated

2 TRPO does not work well on some of tasks compared to DQN

	<i>B. Rider</i>	<i>Breakout</i>	<i>Enduro</i>	<i>Pong</i>	<i>Q*bert</i>	<i>Seaquest</i>	<i>S. Invaders</i>
Random	354	1.2	0	-20.4	157	110	179
Human (Mnih et al., 2013)	7456	31.0	368	-3.0	18900	28010	3690
Deep Q Learning (Mnih et al., 2013)	4092	168.0	470	20.0	1952	1705	581
UCC-I (Guo et al., 2014)	5702	380	741	21	20025	2995	692
TRPO - single path	1425.2	10.8	534.6	20.9	1973.5	1908.6	568.4
TRPO - vine	859.5	34.2	430.8	20.9	7732.5	788.4	450.2

Summary for Part 1

- ① Variants of policy gradient algorithms
- ② Natural Policy Gradient and TRPO
- ③ Next: Other SOTA policy optimization
- ④ Optional homework 3 on policy gradient:
<https://github.com/cuhkrlcourse/ierg6130-assignment/tree/master/assignment3>

ACKTR: Calculating Natural Gradient with KFAC

- 1 Y. Wu, et al. “Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation”. NIPS 2017.
- 2 ACKTR speeds up the optimization by reducing the complexity of calculating the inverse of the Fisher Information Matrix (FIM) using the Kronecker-factored approximation curvature (K-FAC).

$$F = E_{x \sim \pi_{\theta_t}} \left[(\nabla_{\theta} \log \pi_{\theta}(x))^T (\nabla_{\theta} \log \pi_{\theta}(x)) \right]$$

- 1 It is replaced as layer-wise calculation

$$\begin{aligned} F_{\ell} &= \mathbb{E}[\text{vec}\{\nabla_W L\} \text{vec}\{\nabla_W L\}^{\top}] = \mathbb{E}[aa^{\top} \otimes \nabla_s L (\nabla_s L)^{\top}] \\ &\approx \mathbb{E}[aa^{\top}] \otimes \mathbb{E}[\nabla_s L (\nabla_s L)^{\top}] := A \otimes S := \hat{F}_{\ell}, \\ &\quad \text{where } A \text{ denotes } \mathbb{E}[aa^{\top}] \text{ and } S \text{ denotes } \mathbb{E}[\nabla_s L (\nabla_s L)^{\top}]. \end{aligned}$$

Optimizing Neural Networks with Kronecker-factored Approximate Curvature.

- 1 Martens et al. ICML'15: <https://arxiv.org/pdf/1503.05671.pdf>
- 2 Stochastic Gradient Descent (SGD) is the first-order optimization, which is widely used for network optimization; Natural Gradient Descent converges faster in terms of iterations than first-order method, by taking into consideration the curvature of the loss function. However it involves computing the inverse of the Fisher Information Matrix $\theta_{t+1} = \theta_t + \alpha F^{-1} \nabla_{\theta} J(\theta)$, where $F = E_{\pi_{\theta}(s,a)} [\nabla \log \pi_{\theta}(s,a) \nabla \log \pi_{\theta}(s,a)^T]$
- 3 K-FAC (Kronecker-factorized Approximate Curvature) approximate Fisher Information Matrix for large-scale neural network optimization

Optimizing Neural Networks with Kronecker-factored Approximate Curvature.

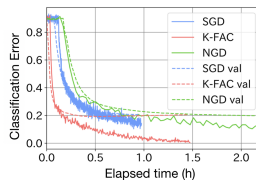
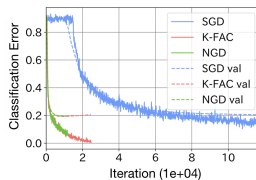
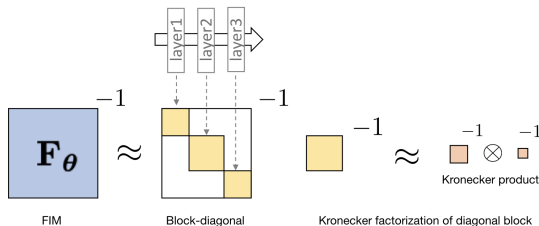
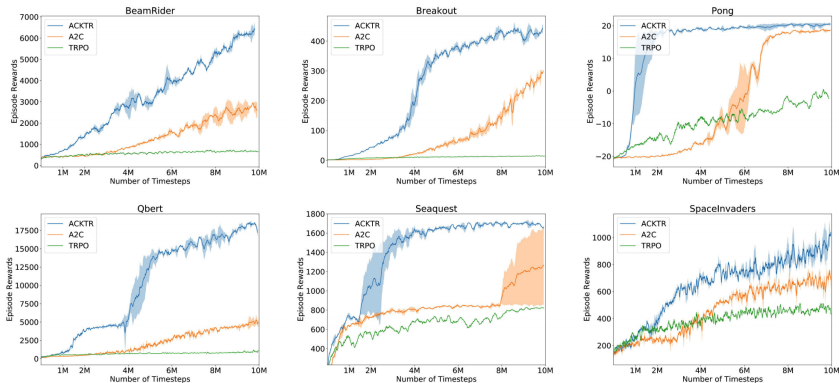


Figure: The comparison of training of ConvNet for CIFAR-10 dataset. Solid line: train, dashed line: validation . Read more on K-FAC tutorial

Performance of ACKTR

1 Performance of ACKTR



2 Introductory link:

<https://blog.openai.com/baselines-acktr-a2c/>

Proximal Policy Optimization (PPO)

- 1 The loss function in the Natural Gradient and TRPO

$$\begin{aligned} & \text{maximize}_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} A_t \right] \\ & \text{subject to } \mathbb{E}_t [KL[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta \end{aligned}$$

- 2 It also can be written into an unconstrained form,

$$\text{maximize}_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} A_t \right] - \beta \mathbb{E}_t [KL[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]]$$

- 3 PPO: include the adaptive KL Penalty, so the optimization will have better insurance that we are optimizing within a trust region

Proximal Policy Optimization (PPO)

Algorithm 4 PPO with Adaptive KL Penalty

Input: initial policy parameters θ_0 , initial KL penalty β_0 , target KL-divergence δ

for $k = 0, 1, 2, \dots$ **do**

 Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

 Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

 Compute policy update

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}(\theta) - \beta_k \bar{D}_{KL}(\theta || \theta_k)$$

 by taking K steps of minibatch SGD (via Adam)

if $\bar{D}_{KL}(\theta_{k+1} || \theta_k) \geq 1.5\delta$ **then**

$$\beta_{k+1} = 2\beta_k$$

else if $\bar{D}_{KL}(\theta_{k+1} || \theta_k) \leq \delta/1.5$ **then**

$$\beta_{k+1} = \beta_k/2$$

end if

end for

- 1 Same performance as TRPO, but solved much faster by first-order optimization (SGD)

PPO with clipping

- 1 Let $r_t(\theta)$ denote the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ so different surrogate objectives:
 - 1 PG without trust region: $L_t(\theta) = r_t(\theta)\hat{A}_t$
 - 2 KL constraint: $L_t(\theta) = r_t(\theta)\hat{A}_t$ s.t. $KL[\pi_{\theta_{old}}, \pi_\theta] \leq \delta$
 - 3 KL penalty: $L_t(\theta) = r_t(\theta)\hat{A}_t - \beta KL[\pi_{\theta_{old}}, \pi_\theta]$
- 2 A new objective function to clip the estimated advantage function if the new policy is far away from the old policy (r_t is too large)

$$L_t(\theta) = \min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t \right)$$

- 1 If the probability ratio between the new policy and the old policy falls outside the range $(1 - \epsilon)$ and $(1 + \epsilon)$, the advantage function will be clipped.
- 2 ϵ is set to 0.2 for the experiments in the PPO paper.

How the clipping works

- 1 Clipping serves as a regularizer by removing incentives for the policy to change dramatically

$$L^{CLIP}(\theta) = \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t, \text{clip}\left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon\right) \hat{A}_t \right)$$

- 2 When the advantage is positive, we encourage the action thus $\pi_{\theta}(a|s)$ increases,

$$L(\theta; \theta_{old}) = \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, (1 + \epsilon) \right) \hat{A}_t$$

- 3 When the advantage is negative, we discourage the action thus $\pi_{\theta}(a|s)$ decreases,

$$L(\theta; \theta_{old}) = \max \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, (1 - \epsilon) \right) \hat{A}_t$$

PPO with clipping

Algorithm 5 PPO with Clipped Objective

Input: initial policy parameters θ_0 , clipping threshold ϵ

for $k = 0, 1, 2, \dots$ **do**

 Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

 Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

 Compute policy update

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

 by taking K steps of minibatch SGD (via Adam), where

$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathbb{E}_{\tau \sim \pi_k} \left[\sum_{t=0}^T \left[\min(r_t(\theta) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k}) \right] \right]$$

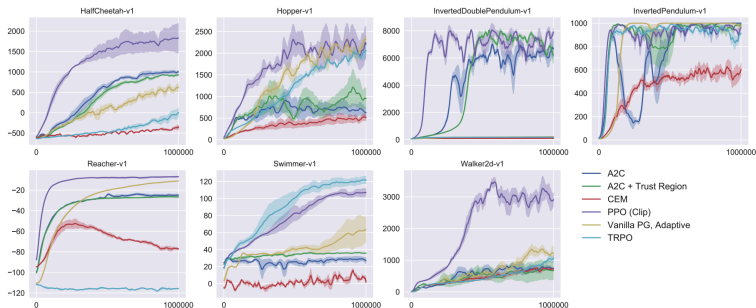
end for

- 1 PPOs have the stability and reliability of trust-region methods but are much simpler to implement, requiring only few lines of code change to a vanilla policy gradient implementation

Result of PPO

① Result on the continuous control tasks (MuJoCo)

<https://gym.openai.com/envs/mujoco>



② Demo of PPO at

<https://blog.openai.com/openai-baselines-ppo/>

③ Emergence of Locomotion Behaviours in Rich Environments by DeepMind (Distributed PPO):

https://www.youtube.com/watch?v=hx_bgoTF7bs

Code of PPO

- 1 Paper link of PPO: <https://arxiv.org/abs/1707.06347>
- 2 Code example: <https://github.com/cuhkrlcourse/DeepRL-Tutorials/blob/master/14.PPO.ipynb>

State of the Art on Policy Optimization

State-of-the-art RL methods are almost all policy-based

- ① **TRPO**: Schulman, L., Moritz, Jordan, Abbeel (2015). Trust region policy optimization
 - ① comment: Solid math proofs and guarantee, but hard to follow
- ② **ACKTR**: Y. Wu, E. Mansimov, S. Liao, R. Grosse, and J. Ba (2017). Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation.
 - ① comment: numeric optimization-based improvement, scalable to real-problems
- ③ **PPO**: Schulman, Wolski, Dhariwal, Radford, Klimov (2017). Proximal policy optimization algorithms
 - ① comment: Easy to read, elegant design of loss function, easy to implement, widely used

Q-learning→DDPG→TD3→SAC

- ① Value based RL methods starting from Q-learning are also being developed over the years
- ② **DDPG**: Deterministic Policy Gradient Algorithms, Silver et al. ICML 2014
- ③ **TD3**: Addressing Function Approximation Error in Actor-Critic Methods, Fujimoto et al. ICML 2018
- ④ **SAC**: Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor, Haarnoja et al. ICML 2018

Deep Deterministic Policy Gradient (DDPG)

- 1 Motivation: how to extend DQN to the environment with continuous action space?
- 2 DDPG is very similar to DQN, which can be considered as a continuous action version of DQN

$$\text{DQN} : a^* = \arg \max_a Q^*(s, a)$$

$$\text{DDPG} : a^* = \arg \max_a Q^*(s, a) \approx Q_\phi(s, \mu_\theta(s))$$

- 1 a deterministic policy $\mu_\theta(s)$ directly gives the action that maximizes $Q_\phi(s, a)$
- 2 as action a is continuous we assume Q-function $Q_\phi(s, a)$ is differentiable with respect to a

Deep Deterministic Policy Gradient (DDPG)

- 1 Thus the DDPG has the following objective:

Q-target: $y(r, s', d) = r + \gamma(1 - d)Q_{\phi_{\text{target}}}(s', \mu_{\theta_{\text{target}}}(s'))$

Q-function: $\min E_{s,r,s',d \sim \mathcal{D}}[Q_{\phi}(s, a) - y(r, s', d)]$

policy: $\max_{\theta} E_{s \sim \mathcal{D}}[Q_{\phi}(s, \mu_{\theta}(s))]$

- 2 Reply buffer and target networks for both value network and policy network are the same as DQN
- 3 DDPG example code (using the same codebase for TD3):
<https://github.com/sfujim/TD3/blob/master/DDPG.py>

Twin Delayed DDPG (TD3)

- 1 One drawback of DDPG is that the learned Q-function sometimes dramatically overestimate Q-values, which then leads to the policy breaking

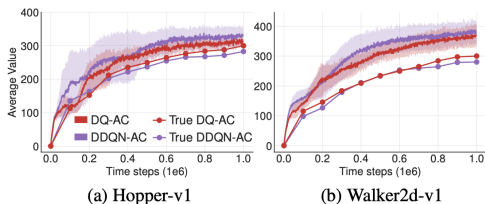


Figure 2. Measuring overestimation bias in the value estimates of actor critic variants of Double DQN (DDQN-AC) and Double Q-learning (DQ-AC) on MuJoCo environments over 1 million time steps.

- 1 true value is estimated using the average discounted return over 1000 episodes following the current policy

Twin Delayed DDPG (TD3)

- ① TD3 introduces three improvements (tricks?)
 - ① **Clipped Double-Q Learning.** TD3 learns two Q-functions instead of one (hence “twin”), and uses the smaller of the two Q-values to form the targets in the Bellman error loss functions.
 - ② **“Delayed” Policy Updates.** TD3 updates the policy (and target networks) less frequently than the Q-function. The paper recommends one policy update for every two Q-function updates.
 - ③ **Target Policy Smoothing.** TD3 adds noise to the target action, to make it harder for the policy to exploit Q-function errors by smoothing out Q along changes in action.

Twin Delayed DDPG (TD3)

- ① TD3 concurrently learns two Q-functions, Q_{ϕ_1} and Q_{ϕ_2} , by mean square Bellman error minimization
- ② Both Q-functions use a single target, calculated using whichever of the two Q-functions gives a smaller value as the following **Q-target**:

$$y(r, s', d) = r + \gamma(1 - d) \min_{i=1,2} Q_{\phi_i, \text{targ}}(s', \mathbf{a}_{\text{TD3}}(s'))$$

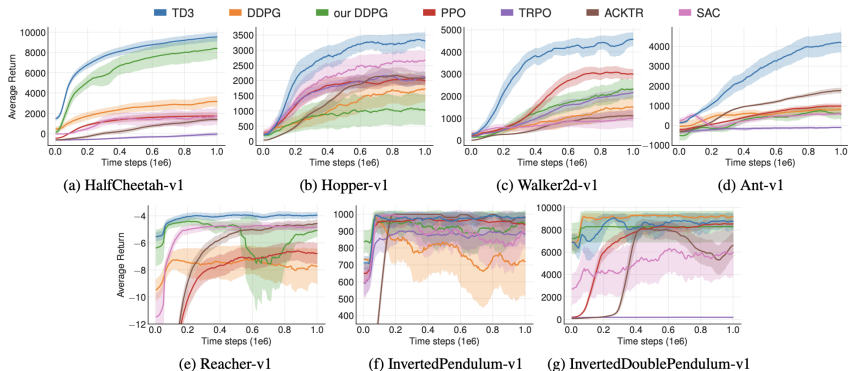
- ③ Target policy smoothing works as follows

$$\mathbf{a}_{\text{TD3}}(s') = \text{clip}(\mu_{\theta, \text{targ}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{low}}, a_{\text{high}}), \epsilon \sim \mathcal{N}(0, \sigma)$$

- ① It serves as a regularizer: to avoid the incorrect sharp peak for some action output by the policy

Twin Delayed DDPG (TD3)

① Result and comparison



Twin Delayed DDPG (TD3)

- ① TD3 paper: Fujimoto, et al. Addressing Function Approximation Error in Actor-Critic Methods. ICML'18:
<https://arxiv.org/pdf/1802.09477.pdf>
- ② Author's Pytorch implementation (**very clean implementation!**):
<https://github.com/sfujim/TD3/>
- ③ Have a comparison to see the difference between DDPG and TD3
 - ① <https://github.com/sfujim/TD3/blob/master/DDPG.py>
 - ② <https://github.com/sfujim/TD3/blob/master/TD3.py>
- ④ **Authors are neither from Berkeley/OpenAI nor DeepMind!**
 - ① Scott Fujimoto, Herke van Hoof, David Meger from McGill University

Soft Actor-Critic (SAC)

- ① SAC optimizes a stochastic policy in an off-policy way, which unifies stochastic policy optimization and DDPG-style approaches
- ② SAC incorporates **entropy regularization**
- ③ Entropy is a quantity which measures how random a random variable is, $H(P) = E_{x \sim P}[-\log P(x)]$
- ④ Entropy-regularized RL: the policy is trained to maximize a trade-off between expected return and entropy, a measure of randomness in the policy

$$\pi^* = \arg \max E_{\tau \sim \pi} \left[\sum_t \gamma^t (R(s_t, a_t, s_{t+1}) + \alpha H(\pi(.|s_t))) \right]$$

Soft Actor-Critic (SAC)

- 1 Value function is changed to include the entropy bonus from every timestep

$$V^\pi(s) = E_{\tau \sim \pi} \left[\sum_t \gamma^t (R(s_t, a_t, s_{t+1}) + \alpha H(\pi(.|s_t))) \mid s_0 = s \right]$$

- 2 The recursive Bellman equation for the entropy-regularized Q^π

$$\begin{aligned} Q^\pi(s, a) &= E_{s' \sim P, a' \sim \pi} [R(s, a, s') + \gamma(Q^\pi(s', a') + \alpha H(\pi(.|s')))] \\ &= E_{s' \sim P, a' \sim \pi} [R(s, a, s') + \gamma(Q^\pi(s', a') - \alpha \log \pi(a'|s'))] \end{aligned}$$

- 3 So the sample update for Q function is

$$Q^\pi(s, a) \approx r + \gamma(Q^\pi(s', \hat{a}') - \alpha \log \pi(\hat{a}'|s')), \hat{a}' \sim \pi(.|s').$$

Soft Actor-Critic (SAC)

- ① SAC concurrently learns a policy π_θ and two Q-functions Q_{ϕ_1} and Q_{ϕ_2}
- ② Like in TD3, the shared target makes use of the clipped double-Q trick and both Q-functions are learned with MSBE minimization

$$L(\phi_i, \mathcal{D}) = E[(Q_{\phi_i}(s, a) - y(r, s', d))^2] \quad (2)$$

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{j=1,2} Q_{\phi_{\text{target},j}}(s', \hat{a}') - \alpha \log \pi_\theta(\hat{a}'|s') \right), \quad (3)$$

$$\hat{a}' \sim \pi_\theta(\cdot|s'). \quad (4)$$

- ③ Policy is learned through maximizing the expected future return plus expected future entropy, thus maximize $V^\pi(s)$

$$\begin{aligned} V^\pi(s) &= E_{a \sim \pi}[Q^\pi(s, a)] + \alpha H(\pi(\cdot|s)) \\ &= E_{a \sim \pi}[Q^\pi(s, a) - \alpha \log \pi(a|s)]. \end{aligned}$$

Soft Actor-Critic (SAC)

- 1 We reparameterize a as sample from a squashed Gaussian policy

$$\hat{a}_\theta(s, \epsilon) = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon), \epsilon \sim \mathcal{N}(0, I).$$

- 2 Reparameterization trick allows us to rewrite the expectation over actions (which contains a pain point: the distribution depends on the policy parameters) into an expectation over noise (which removes the pain point: the distribution now has no dependence on parameters):

$$\begin{aligned} & E_{a \sim \pi_\theta} [Q^{\pi_\theta}(s, a) - \alpha \log \pi_\theta(a|s)] \\ &= E_{\epsilon \sim \mathcal{N}} [Q^{\pi_\theta}(s, \hat{a}_\theta(s, \epsilon)) - \alpha \log \pi_\theta(\hat{a}_\theta(s, \epsilon)|s)] \end{aligned}$$

- 3 The policy is thus optimized as

$$\max_{\theta} E_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [\min_{j=1,2} Q_{\phi_j}(s, \hat{a}_\theta(s, \epsilon)) - \alpha \log \pi_\theta(\hat{a}_\theta(s, \epsilon)|s)] \quad (5)$$

Brief Intro on Reparameterization Trick

- 1 Let's say we want to compute the gradient of the expected value of the function $\nabla_{\theta} E_{x \sim p_{\theta}(x)}[f(x)]$, the difficulty is that x depends on the the distribution with parameter θ
- 2 We can rewrite the samples of the distribution p_{θ} in terms of a noise variable ϵ which is independent of θ

$$\epsilon \sim q(\epsilon) \quad (6)$$

$$x = g_{\theta}(\epsilon) \quad (7)$$

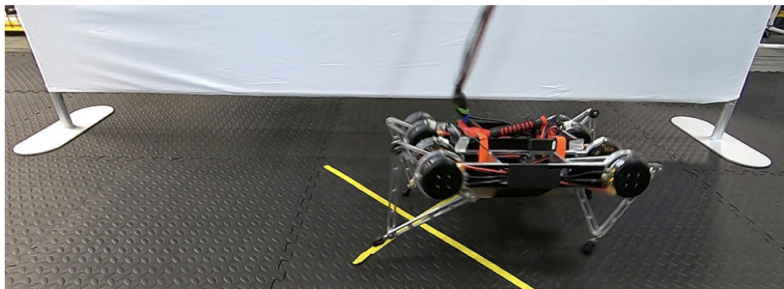
$$\nabla_{\theta} E_{x \sim p_{\theta}(x)}[f(x)] = \nabla_{\theta} E_{\epsilon \sim q(\epsilon)}[f(g_{\theta}(\epsilon))] \quad (8)$$

$$= E_{\epsilon \sim q(\epsilon)}[\nabla_{\theta} f(g_{\theta}(\epsilon))] \quad (9)$$

- 1 where x is reparameterized as a function of ϵ and the stochasticity of p_{θ} is replaced by the distribution $q(\epsilon)$, which could be just a standard Gaussian $\mathcal{N}(0, 1)$, thus $g_{\theta}(\epsilon) = \mu_{\theta} + \epsilon \sigma_{\theta}$ where $\epsilon \sim \mathcal{N}(0, 1)$
- 3 On REINFORCE and Reparameterization: <http://stillbreeze.github.io/REINFORCE-vs-Reparameterization-trick/>

Soft Actor-Critic (SAC)

- 1 Sample code on SAC: <https://github.com/pranz24/pytorch-soft-actor-critic/blob/master/sac.py>
- 2 SAC is known as SOTA for robot learning:
 - 1 Learning to Walk in the Real World with Minimal Human Effort.
<https://arxiv.org/pdf/2002.08550.pdf>
 - 2 <https://www.youtube.com/watch?v=cwyiq6dCg0c>



Second Look at the SOTAs

① Policy Gradient→TRPO→ACKTR→PPO

- ① Stochastic policy thus output probability over discrete actions
- ② Start with policy gradient and importance sampling for off-policy learning

② Q-learning→DDPG→TD3→SAC

- ① Deterministic policy thus output continuous action spaces.
- ② Start with Bellman equation, which doesn't care which transition tuples are used, or how the actions were selected, or what happens after a given transition
- ③ Optimal Q-function should satisfy the Bellman equation for all possible transitions, so very easy for off-policy learning

Tutorial on SOTA RL algorithms

- 1 SpinningUp: Nice implementations and summary of the algorithms from OpenAI: <https://spinningup.openai.com/>



- 2 Stable-baseline: <https://stable-baselines.readthedocs.io/>
 - 1 Currently in TensorFlow
 - 2 PyTorch version is being actively developed:
<https://github.com/hill-a/stable-baselines/issues/733>

